

กลุ่ม 5 — Expressions & Statements

1. คมสัน กลิ่นหอม — Concept + Short Code Illustration
2. จูติพงศ์ ราชธานี — Detailed Code + Live Demo
3. ณัฏฐ์ชนน กุณฑลจิตติวิสัย — Rust vs Other Language + PPL Analysis
4. ณัฏฐพงศ์ อวชัย — Exercises + Common Mistakes + Challenge

Agenda — โครงสร้างการนำเสนอ (รวม ~20 นาที)

แต่ละคนนำเสนอประมาณ 5 นาที ต่อเนื่องกันตามลำดับ

1

Member 1

**Concept + Short Code
Illustration**

แนวคิดหลักของหัวข้อ พร้อมโค้ดตัวอย่างสั้น ๆ

~5 นาที

2

Member 2

Detailed Code + Live Demo

อธิบายโค้ดเชิงลึกและสาธิตการทำงานจริง

~5 นาที

3

Member 3

**Rust vs Other Language +
PPL Analysis**

เปรียบเทียบภาษา และวิเคราะห์เชิง
Programming Language

~5 นาที

4

Member 4

**Exercises + Common
Mistakes + Challenge**

ข้อผิดพลาดที่พบบ่อย และโจทย์ท้าทายผู้ฟัง

~5 นาที

Concept

- Rust เน้นการทำงานผ่าน expressions เป็นหลัก
- Expression คือโค้ดที่ถูกประเมินและให้ค่าออกมา และระหว่างการประเมินอาจทำให้เกิด effect ได้
- Statement คือคำสั่งที่ใช้ประกาศหรือสั่งให้โปรแกรมทำงาน โดยไม่ได้ใช้ผลลัพธ์ต่อ
- แนวคิดนี้เชื่อมกับ Block Expression, return value และการใช้ if / match เป็น Expression

```
fn main() {  
    let x = 5 + 3;  
    println!("{}", x);  
}  
  
// 5 + 3      Expression  
// let x = 5 + 3; Statement  
// 8          Value
```

Expected Output

8

Statement ทั้ง 2 กลุ่ม

- **Declaration Statement** เช่น `let x = 10;` ใช้ประกาศตัวแปร
- **Expression Statement** เช่น `v.pop();` ประเมิน Expression แต่ไม่ใช่ค่าผลลัพธ์ต่อ
- `v.pop()` นำสมาชิกตัวท้ายออกและคืน `Some(3)` แต่ตัวอย่างนี้ทิ้งค่าที่คืนมา
- แม้ไม่ได้ใช้ค่าผลลัพธ์ effect ยังเกิดขึ้น จึงเหลือ `v = [1, 2]`

```
fn main() {  
    let x = 10;  
    let mut v = vec![1, 2, 3];  
  
    v.pop();  
  
    println!("x = {}", x);  
    println!("v = {:?}", v);  
}
```

Expected Output

```
x = 10  
v = [1, 2]
```

Block Expression

- Block { ... } เป็น Expression และมีค่าของตัวเองได้
- ภายใน block มี Statement หลายบรรทัดก่อนสร้างค่าผลลัพธ์ได้
- `a + b` เป็น Expression สุดท้ายและไม่มี `;` จึงทำให้ block มีค่า 8
- ค่าของ block ทั้งก้อนถูกนำไปกำหนดให้ `result`

```
fn main() {  
    let result = {  
        let a = 5;  
        let b = 3;  
        a + b  
    };  
  
    println!("{}", result);  
}
```

Expected Output

8

Tail Expression และ Semicolon

- Tail Expression คือ Expression ท้าย block ที่ไม่มี semicolon (;)
- ค่าของ Tail Expression เป็นค่าของ block ดังนั้น $x = 8$
- เมื่อเติม ; หลัง $5 + 3$ จะกลายเป็น Expression Statement และไม่ใช้ค่า 8 ต่อ
- ถ้า block ไม่มี Tail Expression และจบตามปกติ จะได้ค่า Unit ()

```
fn main() {  
    let x = {  
        5 + 3  
    };  
    let y = {  
        5 + 3;  
    };  
  
    println!("x = {}", x);  
    println!("y = {:?}", y);  
}
```

Expected Output

```
x = 8  
y = ()
```

if Expression

- if ให้ค่าจาก branch ที่ถูกเลือก แล้วนำค่าไปใช้ต่อได้
- ค่า "B" ของ if ทั้งก่อนถูกกำหนดให้ grade
- แต่ละ branch ต้องให้ค่าที่มี **type** เข้ากันได้
ตัวอย่างนี้เป็น &str ทั้งหมด

```
fn main() {  
    let score = 75;  
    let grade = if score >= 80 {  
        "A"  
    } else if score >= 70 {  
        "B"  
    } else {  
        "C"  
    };  
    println!("{}", grade);  
}
```

Expected Output

B

match Expression

- match ให้ค่าจาก arm ที่ตรงกับค่าที่นำมาตรวจ
- arm คือแต่ละกรณี เช่น `2 => 2` ส่วน `_` รองรับกรณีที่เหลือ
- แต่เมื่อ return type ต่างชนิดกันทำให้เกิด compile error

```
fn main() {  
    let number = 2;  
    let text = match number {  
        1 => "One",  
        2 => 2,           //compile error  
        _ => "Other",  
    };  
  
    println!("{}", text);  
}
```

Expected Output

```
error: could not compile
```

loop Expression

- loop เป็น control flow อีกชนิดหนึ่งที่ใช้เป็น Expression และให้ค่าได้
- break 10; หยุด loop และส่งค่า 10 ออกมาเป็นค่าของ loop ทั้งก้อน
- result จึงมีค่า 10 และนำไปใช้งานต่อได้

```
fn main() {  
    let result = loop {  
        break 10;  
    };  
  
    println!("{}", result);  
}
```

Expected Output

```
10
```

Why Expression-Oriented?

เปรียบเทียบการสร้างค่า price

เปลี่ยนค่าภายหลัง

```
let mut price = 0;

if is_member {
    price = 80;
} else {
    price = 100;
}
```

ต้องประกาศ mutable variable ก่อน แล้วเปลี่ยนค่าในแต่ละ branch

สร้างค่าจาก **EXPRESSION** โดยตรง

```
let price = if is_member {
    80
} else {
    100
};
```

กำหนด value จาก if ให้ price โดยตรง ลด mutable state ที่ไม่จำเป็น และติดตามที่มาของค่าได้ง่ายขึ้น

Expression Composition

เปรียบเทียบการสร้างค่า price

- ผลลัพธ์ของ Expression นำไปเป็นส่วนหนึ่งของ Expression อื่นได้โดยตรง
- เมื่อ is_member เป็น true ค่า if คือ 80
- ค่านี้ถูกนำไปคูณกับ quantity ทันที โดยไม่ต้องสร้างตัวแปร price ก่อน
- ดังนั้น $\text{total} = 3 \times 80 = 240$

ONE EXPRESSION INSIDE ANOTHER

```
fn main() {  
    let is_member = true;  
    let quantity = 3;  
  
    let total = quantity * if is_member {  
        80  
    } else {  
        100  
    };  
  
    println!("{}", total);  
}
```

Example 1 การใช้ Block Expression ในการคืนค่า (Return Value)

```
fn main() {
    let score = 85;

    // Block Expression: คืนค่า String slice เข้าตัวแปร grade โดยตรง
    let grade = {
        let bonus = 5;
        let total_score = score + bonus;

        // ไม่ใส่ Semicolon (;) เพื่อให้เป็น Expression คืนค่าออกไป
        if total_score >= 80 {
            "A"
        } else if total_score >= 70 {
            "B"
        } else {
            "F"
        }
    };

    println!("Total calculated grade: {}", grade);
}
```

Demo Checklist

- กำหนดคะแนน score = 85
- คำนวณคะแนนรวม total_score = score + bonus
- ตรวจสอบ Grade ด้วย if-else
- Run โปรแกรม → แสดงผลลัพธ์ A

สิ่งที่ต้องระวังระหว่าง Demo

- ห้ามใส่ ; หลังค่าที่ต้องการ Return
- ตรวจสอบ { } ของ Block Expression ให้ครบ

Example 2 การคืนค่าด้วย match Expression และ return

```
fn check_user_role(level: u32) -> &'static str {
    // Early Return: ใช้คำสั่ง return เพื่อคืนค่าและออกจากฟังก์ชันทันที
    if level == 0 {
        return "Guest";
    }

    // match ในฐานะ Expression: คืนค่า String slice ออกจากฟังก์ชันโดยไม่ต้องใช้คำสั่ง return
    match level {
        1 => "Member",
        2 => "Moderator",
        3 => "Admin",
        _ => "Unknown Role",
    }
}

fn main() {
    let user_level = 2;
    let role = check_user_role(user_level);

    println!("User role is: {}", role);
}
```

Demo Checklist

- คำสั่ง return: คืนค่า "Guest" ออกทันทีเมื่อเงื่อนไขขอบเขต (Early Return)
- match Expression: ประเมินค่าและส่ง Value จาก Arm ที่จับคู่ได้สำเร็จ ออกมาตรง ๆ
- ละเว้น Semicolon (;): ไม่ใช่ ; ทำ match เพื่อให้เป็น Expression สำหรับคืนค่าให้ฟังก์ชัน

สิ่งที่ต้องระวังระหว่าง Demo

- ระวังการใส่ ; ทำโครงสร้าง match เพราะจะทำให้คืนค่าเป็น () (Unit) แทน Value ที่ต้องการ

Example 3 การคืนค่าจาก Loop ด้วยคำสั่ง break

```
fn main() {  
    let mut counter = 0;  
  
    // loop เป็น Expression ที่ส่งค่ากลับมาเข้าตัวแปร result ได้โดยตรง  
    let result = loop {  
        counter += 1;  
  
        if counter == 3 {  
            // คืนค่า counter * 10 ออกไปให้ตัวแปร result แล้วหยุด loop ทันที  
            break counter * 10;  
        }  
    };  
  
    println!("The result from loop execution is: {}", result);  
}
```

Demo Checklist

- loop เป็น Expression: ถือเป็น Expression คืนค่าได้ ไม่ใช่แค่ Statement เหมือนภาษา C/Java
- การใช้ break : คืนค่า counter * 10 ให้ result พร้อมหยุด loop ทันที
- เขียนโค้ดกระชับขึ้น: result ได้รับค่าทันที ไม่ต้องสร้างตัวแปร mut เปล่าๆ ไว้นอก loop

สิ่งที่ต้องระวังระหว่าง Demo

- ระวังการใส่ break; เฉยๆ โดยไม่มีค่าตามหลัง จะได้ค่า () แทน

Rust vs Python

Aspect	Rust	Python
Syntax	ใช้ {} สำหรับ Block และ ; ใช้แยก Statement	ใช้ indentation เพื่อกำหนด Block
Semantics / Behavior	if, loop และ Block สามารถเป็น Expression และคืนค่าได้	if และ loop ใช้ในลักษณะ Statement เป็นหลัก
Type System	Static Type System	Dynamic Type System
Memory Management	ใช้ Ownership และ Borrowing	จัดการ Memory อัตโนมัติ
Safety	Compiler ตรวจสอบ Type และกฎ Ownership/Borrowing	ตรวจสอบ Type หลัก ๆ ขณะ Runtime

Rust vs Python

Rust มีแนวทาง Expression-oriented ที่ทำให้โครงสร้างควบคุมสามารถนำมาใช้สร้างค่าได้โดยตรง

ตัวอย่างนี้แสดงความแตกต่างด้านการออกแบบภาษาอย่างชัดเจน:

Rust:

```
let grade = if score >= 70 {  
    "B"  
} else {  
    "C"  
};
```

if สร้างค่าออกมา แล้วนำค่านั้นไปกำหนดให้ grade

Python:

```
if score >= 70:  
    grade = "B"  
else:  
    grade = "C"
```

if ทำหน้าที่ควบคุมการทำงาน และการกำหนดค่าให้ grade เกิดขึ้นภายในแต่ละ branch

Rust vs Java

Aspect	Rust	Java
Syntax	ใช้ {} สำหรับ Block และ ; สำหรับสิ้นสุด Statement โดย if, match และ Block สามารถเป็น Expression ได้	ใช้ {} สำหรับ Block และ ; สำหรับสิ้นสุด Statement โดย if แบบปกติเป็น Statement
Semantics / Behavior	Expression สามารถประเมินผลและคืนค่าได้ เช่น if สามารถคืนค่าให้ตัวแปรโดยตรง	if แบบปกติใช้ควบคุมการทำงาน และไม่คืนค่าโดยตรง แต่สามารถใช้ Ternary Operator ?: เพื่อสร้างค่าได้
Type System	Static Type System มี Type Checking และ Type Inference	Static Type System มี Type Checking และ Type Inference
Memory Management	ใช้ Ownership, Borrowing และ Lifetime	ใช้ Garbage Collector (GC) จัดการ Memory อัตโนมัติ
Safety	Compiler ตรวจสอบ Type, Ownership และ Borrowing ช่วยป้องกัน Memory Error หลายประเภทตั้งแต่ Compile Time	มี Memory Safety จาก Garbage Collector และไม่มี Pointer ให้จัดการโดยตรง แต่ข้อผิดพลาดบางอย่างเกิดขึ้นได้ตอน Runtime

Rust vs Java

Rust:

```
let grade = if score >= 70 {  
    "B"  
} else {  
    "C"  
};
```

จุดสำคัญคือ if สามารถเป็น Expression และคืนค่า
"B" ให้กับตัวแปร grade ได้

Java:

```
if (score >= 80) {  
    grade = "A";  
} else if (score >= 70) {  
    grade = "B";  
} else {  
    grade = "C";  
}
```

ใน Java if แบบปกติเป็น Statement จึงต้องกำหนดค่าให้
grade ภายในแต่ละ Branch

Rust vs C

Aspect	Rust	Java
Syntax	ใช้ {} สำหรับ Block และ ; สำหรับสิ้นสุด Statement โดย if, match และ Block สามารถเป็น Expression ได้	ใช้ {} สำหรับ Block และ ; สำหรับสิ้นสุด Statement โดย if และ switch เป็น Statement
Semantics / Behavior	Expression สามารถประเมินผลและคืนค่าได้ เช่น if สามารถคืนค่าให้ตัวแปรโดยตรง	if ใช้ควบคุมลำดับการทำงาน และไม่สามารถคืนค่าโดยตรง ต้องกำหนดค่าภายในแต่ละ Branch
Type System	Static Type System มี Type Checking และ Type Inference	Static Type System แต่มี Implicit Conversion และการแปลง Type ที่ยืดหยุ่นกว่า
Memory Management	ใช้ Ownership, Borrowing และ Lifetime	Programmer จัดการ Memory เอง เช่น malloc() และ free()
Safety	Compiler ตรวจสอบ Type, Ownership และ Borrowing ช่วยป้องกัน Memory Error หลายประเภทตั้งแต่ Compile Time	Programmer ต้องรับผิดชอบ Memory Safety เอง จึงมีโอกาสเกิด Memory Leak, Dangling Pointer หรือ Use-after-free

Rust vs C

Rust:

```
let grade = if score >= 70 {  
    "B"  
} else {  
    "C"  
};
```

จุดสำคัญคือ if สามารถเป็น Expression และคืนค่า
"B" ให้กับตัวแปร grade ได้

C:

```
if (score >= 80) {  
    grade = "A";  
} else if (score >= 70) {  
    grade = "B";  
} else {  
    grade = "C";  
}  
  
printf("%s\n", grade);
```

if ทำหน้าที่ควบคุมการทำงาน และการกำหนดค่าให้
grade เกิดขึ้นภายในแต่ละ branch

PPL Perspective — มุมมองเชิง Programming Language

Syntax

ใน Rust, Expression คือโครงสร้างที่สามารถประเมินผลแล้วได้ค่า (Value) ส่วน Statement คือคำสั่งที่ใช้ดำเนินการบางอย่าง โดย Rust ใช้ { } เพื่อกำหนด Block และใช้ ; เพื่อจบ Statement ในหลายกรณี

Semantics

Expression มีหน้าที่ประเมินผลและสร้าง Value
Statement มีหน้าที่ดำเนินการหรือเปลี่ยนแปลงสถานะของโปรแกรม

Type System

Expression ใน Rust ทุกตัวจะมี Type และ Compiler จะตรวจสอบ Type ตั้งแต่ Compile Time เมื่อใช้ **Expression** เพื่อสร้างค่า **Type** ของค่าที่ได้ต้องสอดคล้องกัน โดยเฉพาะเมื่อใช้ **if** เป็น **Expression**

Memory / Resource

Expression และ Statement ใน Rust ทำงานร่วมกับระบบ Ownership, Borrowing และ Lifetime ซึ่งเป็นหัวใจสำคัญในการจัดการ Memory ของภาษา

Abstraction

Function สามารถใช้ซ่อนรายละเอียดการทำงานและรับ Expression เป็น Input/Output

Why Rust?

Rust ออกแบบ Expression & Statements ให้ทำงานร่วมกับ Static Type System และ Ownership System เพื่อให้ได้ทั้ง Safety, Reliability และ Performance

PPL Perspective — มุมมองเชิง Programming Language

Syntax

ใน Rust, Expression คือโครงสร้างที่สามารถประเมินผลแล้วได้ค่า (Value) ส่วน Statement คือคำสั่งที่ใช้ดำเนินการบางอย่าง โดย Rust ใช้ { } เพื่อกำหนด Block และใช้ ; เพื่อจบ Statement ในหลายกรณี ตัวอย่าง:

```
let x = 5 + 3;
```

10 + 5 เป็น Expression เพราะให้ค่า 15

let x = ...; เป็น Statement

Rust มีลักษณะเด่นคือ Expression สามารถอยู่ในตำแหน่งที่ต้องการค่าได้ เช่น if สามารถใช้เป็น Expression ได้

ตัวอย่าง:

```
let result = if x > 0 {  
    1  
} else {  
    -1  
};
```

PPL Perspective — มุมมองเชิง Programming Language

Semantics

Expression มีหน้าที่ประเมินผลและสร้าง Value

Statement มีหน้าที่ดำเนินการหรือเปลี่ยนแปลงสถานะของโปรแกรม

ตัวอย่าง:

```
let x = {  
  let a = 10;  
  a + 5  
};
```

การทำงานคือ

- สร้าง a และกำหนดค่า 10
- ประเมิน a + 5
- ได้ค่า 15
- Block คืนค่า 15
- x จึงมีค่า 15

PPL Perspective — มุมมองเชิง Programming Language

Semantics

อีกจุดสำคัญคือ ; มีผลต่อ Semantics

```
{  
  10 + 5  
}
```

Block นี้คืนค่า 15

```
{  
  10 + 5;  
}
```

Expression ถูกเปลี่ยนให้เป็น Statement และค่าของ Block จะเป็น () หรือ Unit Type

PPL Perspective — มุมมองเชิง Programming Language

Type System

ตัวอย่าง :

```
let x = 10 + 20;
```

Expression $10 + 20$ มี Type เป็นจำนวนเต็ม เช่น int ตามบริบทของการอนุมาน Type

Rust ยังตรวจสอบว่า Expression มี Type ที่สอดคล้องกันหรือไม่

```
let result = if true {  
    10  
} else {  
    20  
};
```

ถูกต้อง เพราะทั้งสอง
Branch ให้ค่า int

```
let result = if true {  
    10  
} else {  
    "Hello"  
};
```

ไม่ถูกต้อง เพราะ
Branch หนึ่งให้ int และ
อีก Branch ให้ string

PPL Perspective — มุมมองเชิง Programming Language

Memory / Resource

Expression และ Statement ใน Rust ทำงานร่วมกับระบบ Ownership, Borrowing และ Lifetime ซึ่งเป็นหัวใจสำคัญในการจัดการ Memory ของภาษา

```
let s1 = String::from("Hello");  
let s2 = s1;
```

เมื่อ s1 ถูกกำหนดให้ s2 จะเกิด Move ทำให้ Ownership ของข้อมูลย้ายไปยัง s2

```
// println!("{}", s1);
```

จะเกิด Compile Error เพราะ s1 ไม่ได้เป็นเจ้าของข้อมูลแล้ว

Rust ยังสามารถใช้ Borrowing:

```
let s = String::from("Hello");  
let len = calculate_length(&s);
```

&s คือการยืมข้อมูลโดยไม่ย้าย Ownership

PPL Perspective — มุมมองเชิง Programming Language

Abstraction

Function สามารถใช้ซ่อนรายละเอียดการทำงานและรับ Expression เป็น Input/Output

```
fn add(a: i32, b: i32) -> i32 {  
    a + b  
}
```

$a + b$ เป็น Expression ที่สร้างผลลัพธ์ให้ Function

PPL Perspective — มุมมองเชิง Programming Language

Why Rust?

Rust ออกแบบ Expression & Statements ให้ทำงานร่วมกับ Static Type System และ Ownership System เพื่อให้ได้ทั้ง Safety, Reliability และ Performance

Safety : Compiler ตรวจสอบ Type, Ownership และ Borrowing ทำให้สามารถตรวจพบปัญหาหลายอย่างก่อน Runtime

Reliability : การบังคับใช้ Type และ Ownership ทำให้พฤติกรรมของโปรแกรมมีความชัดเจนและลดข้อผิดพลาดจากการจัดการข้อมูล

Performance : Rust ไม่มี Garbage Collector จึงสามารถควบคุม Resource ได้อย่างมีประสิทธิภาพ และมี Runtime Overhead ต่ำ เหมาะกับงานที่ต้องการ Performance สูง

Expression-oriented Design การที่ if, match และ Block สามารถคืนค่าได้ ทำให้เขียน Logic ได้กระชับและสามารถนำผลลัพธ์ไปประกอบกับ Expression อื่นได้ทันที

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

ใส่ **Semicolon** หลัง **Expression** สุดท้าย

✗ Incorrect

```
// [โค้ดที่ผิด / ตัวอย่างข้อผิดพลาด]
fn main() {
    let x = {
        5 + 3;
    };
}
```

✓ Correct

```
// [โค้ดที่แก้ไขถูกต้อง]
fn main() {
    let x = {
        5 + 3
    };
}
```

ทำไมถึงผิด? (Why?)

- **สาเหตุ:** เกิดจากผู้เขียนคุ้นเคยกับภาษาที่มักจบคำสั่งด้วย ; จึงเผลอใส่ ; หลัง expression สุดท้ายของ block ทำให้ค่าของ expression นั้นไม่ถูกใช้เป็นค่าของ block
- **หลักการของ Rust ที่เกี่ยวข้อง:** Rust เป็นภาษาแบบ **expression-based** โดย block สามารถเป็น expression และให้ค่าได้จาก expression ตัวสุดท้ายที่ไม่มี ; ต่อท้าย

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

ให้ค่าจาก **if** แต่ละ **branch** เป็นคนละชนิด

✗ Incorrect

```
// [โค้ดที่ผิด / ตัวอย่างข้อผิดพลาด]
fn main() {
    let condition = true;
    let result = if condition {
        10
    } else {
        "ten"
    };
}
```

✓ Correct

```
// [โค้ดที่แก้ไขถูกต้อง]
fn main() {
    let condition = true;
    let result = if condition {
        10
    } else {
        20
    };
}
```

ทำไมถึงผิด? (Why?)

- สาเหตุ: แต่ละ branch ของ `if` คืนค่าคนละ type ทำให้ Rust ไม่สามารถกำหนด type เดียวให้กับผลลัพธ์ของ `if` ได้
- หลักการของ Rust ที่เกี่ยวข้อง: `if` ใน Rust เป็น expression ดังนั้นค่าจากทุก branch ต้องมี type เดียวกันหรือเข้ากันได้

Exercise — Statement หรือ Expression

โจทย์ (Problem)

จงระบุว่าแต่ละบรรทัดเป็น Statement หรือ Expression

```
บรรทัด 1 : let x = 10;
```

```
บรรทัด 2 : x + 5
```

```
บรรทัด 3 : x + 5;
```

```
บรรทัด 4 : if x > 5 { 1 } else { 0 }
```

Exercise — Statement หรือ Expression

โจทย์ (Problem)

จงระบุว่าแต่ละบรรทัดเป็น Statement หรือ Expression

```
บรรทัด 1 : let x = 10;
```

```
บรรทัด 2 : x + 5
```

```
บรรทัด 3 : x + 5;
```

```
บรรทัด 4 : if x > 5 { 1 } else { 0 }
```

💡 Hint

Expression สร้างค่า ส่วน statement ใช้ทำงานบางอย่างและมักจบด้วย ;

Exercise — Statement หรือ Expression

โจทย์ (Problem)

จงระบุว่าแต่ละบรรทัดเป็น Statement หรือ Expression

```
บรรทัด 1 : let x = 10;
บรรทัด 2 : x + 5
บรรทัด 3 : x + 5;
บรรทัด 4 : if x > 5 { 1 } else { 0 }
```

💡 Hint

Expression สร้างค่า ส่วน statement ใช้ทำงานบางอย่างและมักจบด้วย ;

// Solution

ตอบ

```
let x = 10;           -> Statement
x + 5                 -> Expression
x + 5;               -> Expression ที่ถูกใช้เป็น
statement
if x > 5 { 1 } else { 0 } -> Expression
```

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Problem)

- 1.result มีค่าเท่าไร
- 2.if ... elseในโค้ดที่แนบให้เป็น Statement หรือ Expression
- 3.

```
fn main(){
    let x = 4;

    let result = {
        let y = x + 2;

        if y > 5 {
            y * 2
        } else {
            y - 1
        }
    };

    println!("{}", result);
}
```

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Problem)

1. `result` มีค่าเท่าไร
2. `if ... else ...` .ในโค้ดที่แนบให้เป็น Statement หรือ Expression
3. ...

```
fn main(){
    let x = 4;

    let result = {
        let y = x + 2;

        if y > 5 {
            y * 2
        } else {
            y - 1
        }
    };

    println!("{}", result);
}
```

💡 Hint

ลองไล่ค่าจากด้านในออกมาด้านนอก โดยดูว่า `y` มีค่าเท่าไร จากนั้นพิจารณาว่า branch ไหนของ `if` จะถูกเลือก และ expression สุดท้ายของ block คืออะไร

ถ้ามี `;` ต่อท้าย `if` ทั้งหมด ลองคิดว่าค่าของ block ด้านนอกจะยังเป็นตัวเลขอยู่หรือไม่

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Problem)

1. `result` มีค่าเท่าไร
2. `if ... else ...` .ในโค้ดที่แนบให้เป็น Statement หรือ Expression
3. ...

```
fn main(){
    let x = 4;

    let result = {
        let y = x + 2;

        if y > 5 {
            y * 2
        } else {
            y - 1
        }
    };

    println!("{}", result);
}
```

💡 Hint

ลองไล่ค่าจากด้านในออกมาด้านนอก โดยดูว่า `y` มีค่าเท่าไร จากนั้นพิจารณาว่า branch ไหนของ `if` จะถูกเลือก และ expression สุดท้ายของ block คืออะไร
ถ้ามี `;` ต่อท้าย `if` ทั้งหมด ลองคิดว่าค่าของ block ด้านนอกจะยังเป็น ...

// Solution

ตอบ `result = 12` และ `if` เป็น expression ส่วนถ้าเติม `;` หลัง `if` block ด้านนอกจะได้ `()` แทน

Key Takeaways — สรุปสิ่งที่ควรจำ

- 1 **Statement** → ทำงาน แต่ไม่คืนค่า
- 2 **Expression** → ประเมินผลแล้วให้ค่า
- 3 **;** หลัง **Expression** สุดท้ายของ **block** → ทำให้ค่าของ Expression นั้นไม่ถูกใช้เป็นค่าของ block
- 4 ความเชื่อมโยงกับวิชา PPL → เกี่ยวข้องกับ **Syntax & Semantics** ของโค้ด, **Type Checking** ของค่าที่ได้ และ **Control Flow** เช่น `if`

References — เอกสารอ้างอิง

- [The Rust Programming Language – Statements and Expressions](<https://doc.rust-lang.org/book/ch03-03-how-functions-work.html#statements-and-expressions>)
- [Rust By Example – Expressions](<https://doc.rust-lang.org/rust-by-example/expression.html>)
- [The Rust Reference – Statements and Expressions](<https://doc.rust-lang.org/reference/statements-and-expressions.html>)
- [The Rust Reference – Block Expressions](<https://doc.rust-lang.org/reference/expressions/block-expr.html>)
- [Rust Standard Library – Unit Type `()`](<https://doc.rust-lang.org/std/primitive.unit.html>)

GitHub Repository: <https://github.com/670710127/rust-tutorial-2569/tree/main>

Questions & Answers

กลุ่ม [05] — Expressions & Statements